SOFTWARE ENGINEERING · CHAPTER 10

Lab-10: Sequence Diagrams

Continue the ITC Campus Library System from Labs 07 to 09. You already have its use cases (Lab-07), class diagram (Lab-08) and activity diagrams (Lab-09). In five tasks you model *borrow a book* and *return with a fine* as sequence diagrams, add fragments, asynchronous messages and callbacks, derive the REST API and Java signatures from the messages, implement a `LoanService` skeleton with a Mockito `InOrder` test, and finally cross-check every message against the class diagram. The challenge models reservations with timed expiry and generates a sequence diagram automatically from a JUnit test.

UML 2.5.1 Mermaid 11 · PlantUML · draw.io JDK 25 · Spring Boot 4.1.1 · JUnit 6.1 · Mockito 5.23 ≈ 5 hours

5 tasks + 1 challenge

How to work through this lab. The tasks build on each other inside *one* model repository. For every diagram submit the **source text** (`.mmd` Mermaid, `.puml` PlantUML or `.drawio`) *and* a PNG or SVG export. Commit after every task (`git tag task-N`). Tick the acceptance checklist as you go (it is saved in your browser). Open hints only after 10 minutes of being stuck.

0 Setup — 15 min

1. Verify the toolchain and a diagram renderer (any one of the three):

```
java -version                # 25
mvn -v                       # Apache Maven 3.9.x (or ./mvnw)
npx -y @mermaid-js/mermaid-cli -V          # Mermaid CLI 11.x (renders .mmd
to PNG/SVG)
java -jar plantuml.jar -version            # alternative: PlantUML
# or draw.io desktop: File > Export as > PNG (tick "Include a copy of my
diagram")
```

2. Open your `library-model` repository from Lab-09 and create the folder for this lab:

```
library-model/
├── diagrams/
│   ├── 07-usecases/  08-classes/  09-activities/       ← from earlier labs
│   └── 10-sequence/                                    ← this lab
```

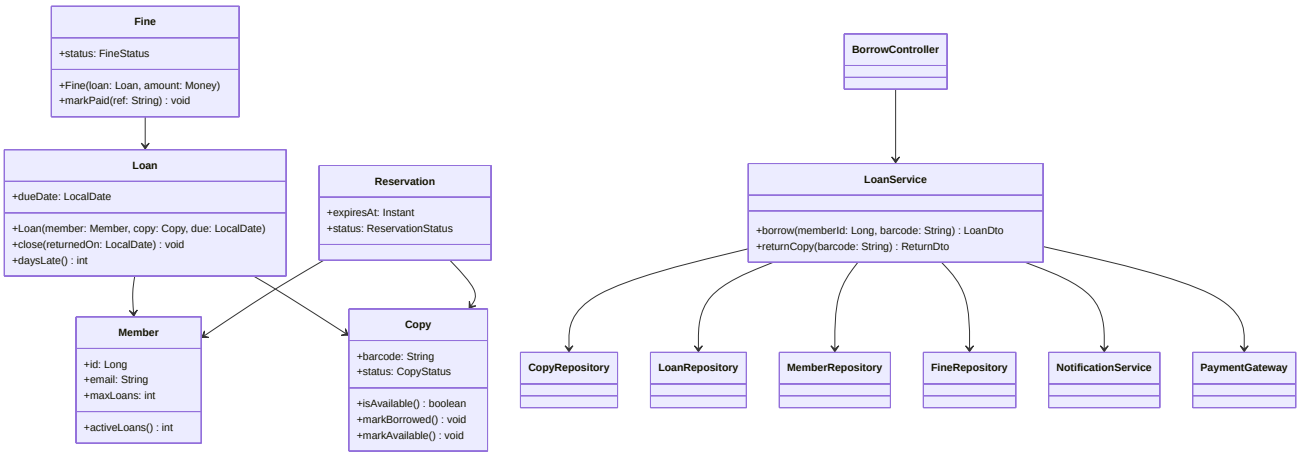
```
│       ├── t1-borrow-happy.mmd          t1-borrow-happy.png
│       ├── t2-borrow-fragments.mmd      t2-borrow-fragments.png
│       ├── t3-return-with-fine.mmd      t3-return-with-fine.png
│       ├── t4-borrow-ecb.mmd            t4-borrow-ecb.png
│       ├── t5-borrow-errors.mmd         t5-borrow-errors.png
│       └── challenge-reservation.mmd    challenge-reservation.png
├── docs/
│   ├── api-10.md                  ← Task 4: REST API derived from the
messages
│   └── consistency-10.md          ← Task 5: consistency table
└── library/                       ← Spring Boot project (Task 4 skeleton +
tests)
```

3. If you do not have the Spring Boot project yet, generate it (the same coordinates as Lab-06):

```
curl https://start.spring.io/starter.zip \
  -d type=maven-project -d language=java -d javaVersion=25 \
  -d bootVersion=4.1.1 -d groupId=edu.itc.se -d artifactId=library \
  -d packageName=edu.itc.library \
  -d dependencies=webmvc,validation,data-jpa,h2 \
  -o library.zip && unzip library.zip -d library
```

`spring-boot-starter-test` (JUnit 6.1, AssertJ, Mockito 5.23) is included automatically.

4. Reference: the classes from your Lab-08 class diagram that this lab uses. If your names differ, use *your* names consistently, and say so in `docs/consistency-10.md`.



5. Render one test diagram to make sure the toolchain works: `npx @mermaid-js/mermaid-cli -i diagrams/10-sequence/t1-borrow-happy.mmd -o diagrams/10-sequence/t1-borrow-happy.png -b white`.

1 Borrow a book: the happy path Easy 30 min

GOAL

Draw your first sequence diagram: the main success scenario of use case *UC-2 Borrow a book* from Lab-07, with correct lifelines, activation bars, synchronous calls and replies, and message names that are operations of the Lab-08 classes.

STEPS

1. Open UC-2 from Lab-07 and list its main-scenario steps (typically: member scans card and copy, system checks the member, system checks the copy, system records the loan, system shows the due date).
2. Create `diagrams/10-sequence/t1-borrow-happy.mmd` with exactly these participants, in this order: `actor Member`, `BorrowController`, `LoanService`, `MemberRepository`, `CopyRepository`, `LoanRepository`.
3. Add the first message from the actor to the boundary: `POST /api/loans (memberId, barcode)`, then `borrow(memberId, barcode)` from the controller to the service. Activate both receivers with `+`.
4. From `LoanService` add, in order: `findById(memberId)`, `findByBarcode(barcode)`, `isAvailable()` on the copy (draw `copy : Copy` as a lifeline created by the repository reply, or as a self-call on the service if you prefer, and justify the choice in a Note), `markBorrowed()`, `save(loan)`. Every call that returns data gets a dashed reply labelled with the value.
5. Close every activation with a reply (`-`): `LoanDto(dueDate)` back to the controller, `201 Created` back to the member.
6. Render the PNG and check that no reply arrow ends on a lifeline other than the caller.
7. Add a frame name as a Note or as the first line comment: `%% sd BorrowBook (UC-2 main scenario)`. Commit and tag `task-1`.

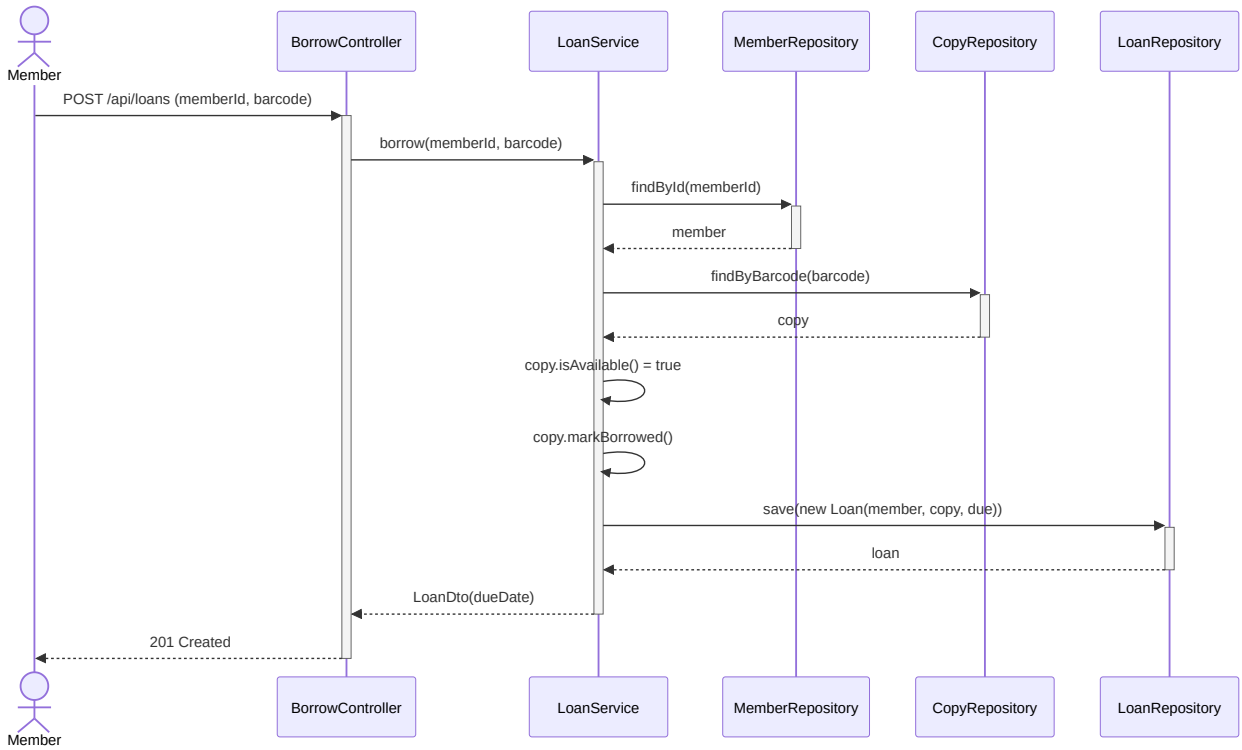
STARTER DIAGRAM TEXT

```
%% sd BorrowBook (UC-2 main scenario)
sequenceDiagram
  actor M as Member
  participant BC as BorrowController
  participant LS as LoanService
  participant MR as MemberRepository
  participant CR as CopyRepository
  participant LR as LoanRepository
  M->>+BC: POST /api/loans (memberId, barcode)
  BC->>+LS: borrow(memberId, barcode)
  LS->>+MR: findById(memberId)
  MR-->>-LS: member
  %% TODO: findByBarcode, isAvailable, markBorrowed, save(loan)
```

```
    LS-->>-BC: LoanDto(dueDate)
    BC-->>-M: 201 Created
```

EXPECTED OUTPUT

A diagram with six lifelines (plus optionally `copy : Copy`), nine to eleven messages, four activation bars that all close, and replies that carry `member`, `copy`, `true`, `loan`, `LoanDto(dueDate)`, `201 Created`. Roughly:



Show hints

- Every message label must be a method of the receiver's class in your Lab-08 diagram. If `CopyRepository` has no `findByBarcode`, add it to the class diagram as well (and note the change for Task 5).
- Calling an operation on an object the service already holds (`copy.isAvailable()`) can be drawn as a message to a `copy : Copy` lifeline; Mermaid's `create participant` is not needed because the copy already exists in the database.
- Do not draw the database: the repository is the last lifeline. SQL is an implementation detail of the repository.
- `+`/`-` must balance per participant; Mermaid reports "inactive participant" when a `-` has no matching `+`.

ACCEPTANCE CHECKLIST

- [ ] Participants are in first-use order; the actor is a stick figure; no database lifeline.
- [ ] Every activation opened with `+` is closed with a reply; replies return to the caller and carry the returned value.

- [ ] Every message name is an operation of the receiver's class in the Lab-08 diagram (or the class diagram was updated).

- [ ] `t1-borrow-happy.mmd` and `.png` committed and tagged `task-1`.

2 Fragments: alt, loop and opt **Medium** **40 min**

GOAL

Extend the happy path with the extensions of UC-2: the member may be over the loan limit, a copy may not be available, several copies can be borrowed in one visit, and a reservation holder may need to be notified.

STEPS

1. Copy `t1-borrow-happy.mmd` to `t2-borrow-fragments.mmd`. Change the request to carry a list: `POST /api/loans (memberId, barcodes[])`.
2. After `findById(memberId)` add an `alt` with guard `member.activeLoans() >= member.maxLoans`: the service replies `throw LoanLimitExceededException` and the controller answers `409 Conflict`; the `else` operand contains the rest of the scenario.
3. Wrap the per-copy work in `loop for each barcode in barcodes`: `findByBarcode`, availability check, `markBorrowed`, `save(loan)`.
4. Inside the loop add a second `alt`: guard `copy.isAvailable()` for the normal path, `else` for `throw CopyNotAvailableException(barcode)`. Decide (and write in a Note) whether one unavailable copy aborts the whole request or only skips that copy, and match the Lab-09 activity diagram.
5. After the loop add `opt reservation exists for copy`: `findActiveByCopy(copy)` on a `ReservationRepository` lifeline, then `notifyReservationCancelled(reservation)` to `NotificationService`.
6. Render, then compare every guard with the decision nodes of your Lab-09 activity diagram for UC-2; the guard texts must match word for word. Commit and tag `task-2`.

STARTER DIAGRAM TEXT

```
sequenceDiagram
  actor M as Member
  participant BC as BorrowController
  participant LS as LoanService
  participant MR as MemberRepository
  participant CR as CopyRepository
  participant LR as LoanRepository
```

```
  participant RR as ReservationRepository
  participant NS as NotificationService
  M->>+BC: POST /api/loans (memberId, barcodes[])
  BC->>+LS: borrow(memberId, barcodes)
  LS->>+MR: findById(memberId)
  MR-->>-LS: member
  alt member.activeLoans() >= member.maxLoans
    LS-->>BC: throw LoanLimitExceededException
    BC-->>M: 409 Conflict
  else within limit
    loop for each barcode in barcodes
      LS->>+CR: findByBarcode(barcode)
      CR-->>-LS: copy
      alt copy.isAvailable()
        %% TODO markBorrowed, save(loan)
      else not available
        %% TODO throw CopyNotAvailableException(barcode)
      end
    end
    opt reservation exists for copy
      %% TODO findActiveByCopy, notifyReservationCancelled
    end
    LS-->>-BC: List of LoanDto
    BC-->>-M: 201 Created
  end
```

EXPECTED OUTPUT

```
Fragments present (top to bottom):
  alt  [member.activeLoans() >= member.maxLoans]  /  [else]
    loop [for each barcode in barcodes]
      alt [copy.isAvailable()]  /  [else]
    opt  [reservation exists for copy]
Guards are boolean expressions the LoanService can evaluate.
Two new lifelines: ReservationRepository, NotificationService.
Every failure operand ends at the Member with an HTTP status (409).
```

Show hints

- Mermaid adds the square brackets around the guard itself: write `alt copy.isAvailable()`, not `alt [copy.isAvailable()]`.
- An exception is a reply arrow labelled `throw X`. If you draw it, the activation of the service ends there; do not add a second normal reply in the same operand.
- If a failed copy aborts the request, the second `alt` is really a `break` (Chapter 10, Topic 9). Either is acceptable if the Note explains it and Lab-09 agrees.

- Keep the diagram under 20 messages; if it grows beyond that, move the reservation notification to its own diagram and reference it with a Note `ref: NotifyReservation`.

ACCEPTANCE CHECKLIST

- [ ] Two `alt` fragments with exhaustive, boolean guards and an `else` operand each.
- [ ] A `loop` over the requested barcodes containing the per-copy messages, and an `opt` for the reservation notification.
- [ ] Guards match the decision nodes of the Lab-09 activity diagram word for word; the abort-or-skip decision is documented in a Note.
- [ ] Both failure operands reach the Member with an HTTP status; source and PNG committed, tag `task-2`.

3

Async messages, object creation and callbacks: return with a fine

Medium **45 min**

GOAL

Model *UC-3 Return a copy* when the copy is late: a `Fine` object is created, the member is notified asynchronously, the copy is released in parallel, and the fine is later settled through an online payment whose provider calls our webhook.

STEPS

1. Create `t3-return-with-fine.mmd` with participants `actor Member`, `ReturnController`, `LoanService`, `CopyRepository`, `LoanRepository`, `FineRepository`, `NotificationService`, `PaymentGateway`, `actor Payment provider`, `PaymentWebhookController`.
2. Happy start: `POST /api/returns (barcode)` → `returnCopy(barcode)` → `findByBarcode` → `findOpenByCopy(copy)` → `loan.close(today)` (self-call or message to `loan : Loan`).
3. Add `alt loan.daysLate() > 0`. Inside: `create participant fine : Fine` followed by `new Fine(loan, amount)`, then `save(fine)` on `FineRepository`.
4. Still inside the operand add a `par`: operand 1 is the **asynchronous** message `sendFineNotice(fine)` to `NotificationService` (`-)` arrow, no reply); operand 2 is `copy.markAvailable()` and `save(copy)`. The `else` operand releases the copy only.
5. Close the request: `ReturnDto(fineAmount)` and `200 OK`. Then add a Note "later, from the fine notice e-mail" and the payment half: `POST /api/fines/{id}/pay` → `payFine(id,`

`card)` → `charge(card, amount)` to `PaymentGateway` → reply `PaymentResult(PENDING, ref)` → `202 Accepted`.

6. Add the callback: `Payment provider -) PaymentWebhookController: POST /webhooks/payments (ref, status)`, then `onFinePaid(ref)` to `LoanService`, `fine.markPaid(ref)`, `200 OK` back to the provider. The callback must start its own activation and must not be drawn as the reply of `charge`.
7. Write the Java that the arrows stand for (below) in `library/src/main/java/edu/itc/library/`: constructor `Fine(Loan, Money)`, `@Async NotificationService.sendFineNotice`, `PaymentWebhookController`. It only needs to compile. Commit and tag `task-3`.

STARTER CODE

```
package edu.itc.library.notification;

@Service
public class NotificationService {
    private final JavaMailSender mail;
    public NotificationService(JavaMailSender mail) { this.mail = mail; }

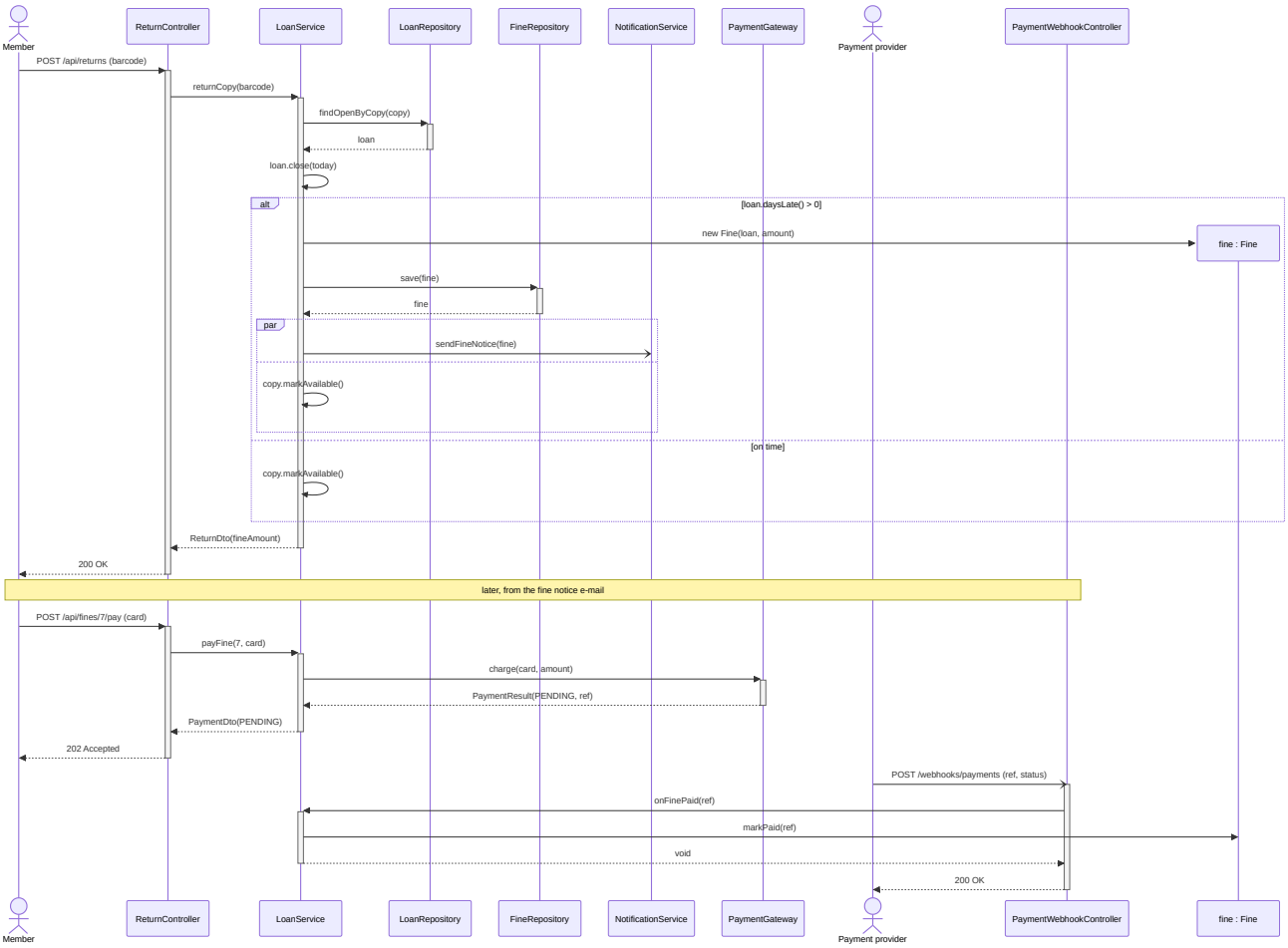
    // asynchronous message: open arrow head, no reply in the diagram
    @Async
    public CompletableFuture<Void> sendFineNotice(Fine fine) {
        // TODO build and send the message; return a completed future
        return CompletableFuture.completedFuture(null);
    }
}

// «create» message: new Fine(loan, amount)
@Entity
public class Fine {
    @Id @GeneratedValue private Long id;
    @ManyToOne private Loan loan;
    private BigDecimal amount;
    private FineStatus status = FineStatus.OPEN;
    private String paymentRef;
    protected Fine() { }
    public Fine(Loan loan, Money amount) { /* TODO */ }
    public void markPaid(String ref) { /* TODO idempotent */ }
}

@RestController
@RequestMapping("/webhooks/payments")
public class PaymentWebhookController {
    private final LoanService loanService;
    public PaymentWebhookController(LoanService s) { this.loanService = s; }
```

```
    // callback from the provider: a NEW message, on a new request thread
    @PostMapping
    public ResponseEntity<Void> onEvent(@RequestBody PaymentEvent event) {
        // TODO verify signature, then loanService.onFinePaid(event.ref())
        return ResponseEntity.ok().build();
    }
}
```

EXPECTED OUTPUT



Show hints

- In Mermaid the line after `create participant X` must be a message *to* X. Put the constructor call there.
- Two lifelines for the same class are fine when the diagram grows (`CopyRepository` was dropped above to keep it under 30 messages); say in a Note what you left out.
- `@Async` needs `@EnableAsync` on a configuration class; without it the arrow would secretly be synchronous.
- The webhook is retried by providers: `markPaid` must ignore a second call with the same `ref`. A `Note over WH` saying "idempotent" is worth points.

- The two request halves happen minutes apart. A Note across all lifelines is the standard way to show a time gap; UML also allows a duration constraint `{t < 24h}`.

ACCEPTANCE CHECKLIST

- [ ] `fine : Fine` is created with a create message at the right point in time; the constructor exists in Java.
- [ ] `sendFineNotice` is an asynchronous message with an open arrow head and no reply; the Java method is `@Async` and returns a `CompletableFuture`.
- [ ] A `par` fragment separates notification from releasing the copy; the webhook callback is a new message with its own activation, not a reply.
- [ ] `PaymentWebhookController`, `Fine` and `NotificationService` compile (`./mvnw -q compile`); source and PNG committed, tag `task-3`.

4 ECB analysis and API design **Medium** **60 min**

GOAL

Redraw the borrow scenario with «boundary», «control» and «entity» stereotypes, derive the REST API and the Java method signatures from the messages, implement the `LoanService` skeleton and prove with a Mockito `InOrder` test that the code calls its collaborators in the order of the diagram.

STEPS

1. Copy `t2-borrow-fragments.mmd` to `t4-borrow-ecb.mmd`. Prefix every participant label with its stereotype: `«boundary» BorrowController`, `«control» LoanService`, `«entity» MemberRepository`, and so on. Check the three ECB rules; if the controller talks to a repository anywhere, move that message into the service.
2. Write `docs/api-10.md` with one table row per *message that enters a boundary* across Tasks 1 to 3: message, HTTP method and path, request body, success status and body, failure statuses. Expected rows: `POST /api/loans`, `POST /api/returns`, `POST /api/fines/{id}/pay`, `POST /webhooks/payments`, plus `GET /api/loans/{id}` for the query.
3. Below the table list the Java signatures derived from the messages into `LoanService`: `borrow(Long memberId, List<String> barcodes): List<LoanDto>`, `returnCopy(String barcode): ReturnDto`, `payFine(Long fineId, Card card): PaymentDto`,

`onFinePaid(String ref): void`. Every parameter comes from a message argument, every return type from a reply.

4. Implement `LoanService.borrow` in `library/src/main/java/edu/itc/library/loan/LoanService.java` exactly following `t4-borrow-ecb.mmd`: same collaborators, same order, `if` for each `alt`, `for` for the `loop`. Other methods may stay `TODO`.
5. Write `LoanServiceTest` with `@ExtendWith(MockitoExtension.class)`: one `@Mock` per lifeline, one `when` per reply, one `InOrder.verify` per message, `verifyNoMoreInteractions` at the end. One test method per `alt` operand: `borrowsInDiagramOrder`, `rejectsMemberOverLimit`, `rejectsUnavailableCopy`.
6. Run `./mvnw -q test`. Then deliberately swap two calls in `borrow` and confirm the `InOrder` test fails; revert. Commit and tag `task-4`.

STARTER CODE

```
package edu.itc.library.loan;

@Service                                                // «control»
public class LoanService {
    private final MemberRepository members;          // «entity» gateways
    private final CopyRepository copies;
    private final LoanRepository loans;
    private final ReservationRepository reservations;
    private final NotificationService notifications;
    private final Clock clock;

    public LoanService(MemberRepository members, CopyRepository copies,
                       LoanRepository loans, ReservationRepository reservations,
                       NotificationService notifications, Clock clock) {
        this.members = members; this.copies = copies; this.loans = loans;
        this.reservations = reservations;
        this.notifications = notifications; this.clock = clock;
    }

    @Transactional
    public List<LoanDto> borrow(Long memberId, List<String> barcodes) {
        Member member = members.findById(memberId)
            .orElseThrow(() -> new MemberNotFoundException(memberId));
        // alt member.activeLoans() >= member.maxLoans
        if (member.activeLoans() >= member.maxLoans()) {
            throw new LoanLimitExceededException(memberId);
        }
        var result = new ArrayList<LoanDto>();
        // loop for each barcode in barcodes
        for (String barcode : barcodes) {
```

```
            Copy copy = copies.findByBarcode(barcode)
                .orElseThrow(() -> new CopyNotFoundException(barcode));
            // alt copy.isAvailable()
            // TODO: markBorrowed, save(new Loan(...)), add LoanDto; else throw
        }
        // opt reservation exists for copy
        // TODO: reservations.findActiveByCopy(copy).ifPresent(
        //           r -> notifications.notifyReservationCancelled(r));
        return result;
    }
}


@ExtendWith(MockitoExtension.class)
class LoanServiceTest {
    @Mock MemberRepository members;
    @Mock CopyRepository copies;
    @Mock LoanRepository loans;
    @Mock ReservationRepository reservations;
    @Mock NotificationService notifications;
    Clock clock = Clock.fixed(Instant.parse("2026-09-25T10:00:00Z"),
ZoneOffset.UTC);
    LoanService service;

    @BeforeEach
    void setUp() {
        service = new LoanService(members, copies, loans, reservations,
                                  notifications, clock);
    }

    @Test
    void borrowsInDiagramOrder() {
        Member member = member(1L, 0, 5);
        Copy copy = availableCopy("C-100");
        when(members.findById(1L)).thenReturn(Optional.of(member));
        when(copies.findByBarcode("C-100")).thenReturn(Optional.of(copy));
        when(loans.save(any(Loan.class))).thenAnswer(inv -> inv.getArgument(0));
        when(reservations.findActiveByCopy(copy)).thenReturn(Optional.empty());

        List<LoanDto> dtos = service.borrow(1L, List.of("C-100"));

        InOrder seq = inOrder(members, copies, loans, reservations,
notifications);
        seq.verify(members).findById(1L);
        seq.verify(copies).findByBarcode("C-100");
        seq.verify(loans).save(any(Loan.class));
        seq.verify(reservations).findActiveByCopy(copy);
        seq.verifyNoMoreInteractions();
```

```
        assertThat(dtos).hasSize(1);
        assertThat(dtos.getFirst().dueDate()).isEqualTo(LocalDate.of(2026, 10,
9));
    }

    // TODO rejectsMemberOverLimit(): expects LoanLimitExceededException and
    //      verifies that copies and loans were never touched
    // TODO rejectsUnavailableCopy(): expects CopyNotAvailableException
}
```

EXPECTED OUTPUT

```
$ ./mvnw -q test
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0

docs/api-10.md (excerpt)
| Message into boundary      | Endpoint                 | Body               |
Success        | Failures      |
|----------------------------|--------------------------|--------------------|--
--------------|---------------|
| borrow(memberId, barcodes) | POST /api/loans          | memberId, barcodes |
201 [LoanDto]  | 400, 404, 409 |
| returnCopy(barcode)        | POST /api/returns        | barcode            |
200 ReturnDto  | 404, 409      |
| payFine(fineId, card)      | POST /api/fines/{id}/pay | card               |
202 PaymentDto | 404, 422, 503 |
| onFinePaid(ref)            | POST /webhooks/payments  | ref, status, sig   |
200            | 401           |
| findById(id)  (query)      | GET /api/loans/{id}      | -                  |
200 LoanDto    | 404           |

After swapping copies.findByBarcode and members.findById in borrow():
[ERROR] borrowsInDiagramOrder  Wanted but not invoked: members.findById(1L)
        ... Wanted anywhere AFTER following interaction:
copies.findByBarcode("C-100")
```

Show hints

- Inject a `Clock` so the due date (`today + 14 days`) is deterministic in the test; register a `Clock.systemUTC()` bean in a configuration class.
- `verifyNoMoreInteractions()` fails if the code calls something the diagram does not show, such as `copies.save(copy)`. Either add the message to the diagram or rely on JPA dirty checking and leave it out. Both are defensible; the diagram and the code must agree.
- Mockito 5 strict stubs: an unused `when(...)` fails the test with `UnnecessaryStubbingException`. Stub only replies that the scenario actually uses.

- In `rejectsMemberOverLimit` use `verifyNoInteractions(copies, loans)`: the diagram shows no message to them in that operand.
- Mermaid accepts the guillemets in participant labels: `participant LS as «control» LoanService`.

ACCEPTANCE CHECKLIST

- [ ] Every lifeline in `t4-borrow-ecb.mmd` carries a stereotype and the three ECB rules hold (no boundary → entity, no entity → boundary).
- [ ] `docs/api-10.md` lists five endpoints with statuses and the four `LoanService` signatures derived from messages and replies.
- [ ] `LoanService.borrow` mirrors the diagram (same collaborators, same order, `if`/`for` for `alt`/`loop`) and contains no HTTP or persistence code.
- [ ] Three Mockito tests pass; the `InOrder` test fails when two calls are swapped; tag `task-4`.

5 Error flows and consistency with the class diagram Hard

60 min

GOAL

Model three failure scenarios end to end, from the exception in the control to the `ProblemDetail` the member receives, then cross-check every message of Tasks 1 to 4 against the Lab-08 class diagram and fix the mismatches on the correct side.

STEPS

1. Create `t5-borrow-errors.mmd` with participants `actor Member`, `BorrowController`, `LoanService`, `MemberRepository`, `CopyRepository`, `ApiExceptionHandler`. Model three scenarios in one diagram, each in its own operand (`alt` with three guards, or three `break` fragments): **(a)** member not found, **(b)** copy not available, **(c)** member over the loan limit.
2. In each operand: the reply from `LoanService` is `throw XException`, then `BorrowController` (through the `DispatcherServlet`, mention it in a Note) sends `handle(XException)` to `ApiExceptionHandler`, gets `ProblemDetail(status, detail)` back and answers the member with the status and `application/problem+json`.
3. Fill a table in `docs/api-10.md`: scenario, exception class, HTTP status (404 / 409 / 409), `title`, `detail` template, extra properties (`memberId`, `barcode`, `maxLoans`). Implement `ApiExceptionHandler` with one `@ExceptionHandler` per exception returning a `ProblemDetail`; no `try/catch` in the controller.

4. Add a `@WebMvcTest(BorrowController.class)` test with `@MockitoBean LoanService` that stubs `borrow` to throw `CopyNotAvailableException` and asserts status 409, content type `application/problem+json` and `$.barcode`.
5. Consistency check: for every message in `t1` to `t5` (and the messages of Task 3) fill one row of `docs/consistency-10.md`: message, sender, receiver, operation present in the Lab-08 class diagram (yes/no), reply type equals the operation's return type (yes/no), association sender → receiver exists (yes/no), fix. Expect 25 to 40 rows.
6. Fix every "no": either add the operation, return type or association to the class diagram (update `diagrams/08-classes/` with a comment `# changed in Lab-10`), or rename or move the message in the sequence diagram. Also list the guards of your Lab-09 activity diagram that had no matching `alt` operand. Re-render all PNGs, commit and tag `task-5`.

STARTER CODE

```
package edu.itc.library.web;

@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(CopyNotAvailableException.class)
    ProblemDetail copyNotAvailable(CopyNotAvailableException ex) {
        var pd = ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT,
            "Copy " + ex.barcode() + " is not available");
        pd.setTitle("Copy not available");
        pd.setProperty("barcode", ex.barcode());
        return pd;
    }

    // TODO MemberNotFoundException -> 404, LoanLimitExceededException -> 409
}
```

```
%% t5-borrow-errors.mmd (skeleton)
sequenceDiagram
  actor M as Member
  participant BC as BorrowController
  participant LS as LoanService
  participant MR as MemberRepository
  participant CR as CopyRepository
  participant EH as ApiExceptionHandler
  M->>+BC: POST /api/loans (memberId, barcodes[])
  BC->>+LS: borrow(memberId, barcodes)
  LS->>+MR: findById(memberId)
  MR-->>-LS: Optional.empty()
  break member not found
    LS-->>-BC: throw MemberNotFoundException(memberId)
```

```
    BC->>+EH: handle(MemberNotFoundException)
    EH-->>-BC: ProblemDetail(404, "Member 42 not found")
    BC-->>-M: 404 Not Found (application/problem+json)
  end
  %% TODO scenarios (b) copy not available -> 409 and (c) over limit -> 409
```

EXPECTED OUTPUT

```
$ curl -i -X POST localhost:8080/api/loans -H 'Content-Type: application/json' \
       -d '{"memberId":1,"barcodes":["C-100"]}'
HTTP/1.1 409
Content-Type: application/problem+json

{"type":"about:blank","title":"Copy not available","status":409,
 "detail":"Copy C-100 is not available","instance":"/api/loans","barcode":"C-
100"}

docs/consistency-10.md (excerpt)
| #  | Message -> Receiver                        | Op  | Reply | Assoc | Fix
|
|----|--------------------------------------------|-----|-------|-------|--------
--------------------------|
| 1  | borrow(memberId, ...) -> LoanService       | yes | yes   | yes   | -
|
| 5  | findByBarcode(barcode) -> CopyRepository   | no  | -     | yes   | add
findByBarcode(String) to CD  |
| 9  | markBorrowed() -> Copy                     | yes | yes   | no    | add
LoanService ..> Copy to CD   |
| 14 | sendFineNotice(fine) -> NotificationSvc    | no  | n/a   | yes   | rename
notify() in CD            |
| 17 | markPaid(ref) -> Fine                      | yes | yes   | no    |
LoanService ..> Fine or via repo |
Summary: 31 messages, 7 mismatches, 5 fixed in the class diagram, 2 fixed in the
sequence diagrams.
```

Show hints

- 404 for a missing member, 409 for a state conflict (copy borrowed, limit reached). 422 is for a well-formed request that cannot be processed for business reasons; either 409 or 422 is accepted for the limit if the choice is argued in `api-10.md`.
- Three `break` fragments in sequence read better than one `alt` with three guards, because each one exits the interaction. Both are valid UML.
- In the MockMvc test use `MediaType.APPLICATION_PROBLEM_JSON` and `jsonPath("$.status").value(409)`; Spring Boot 4 uses `@MockitoBean`, not `@MockBean`.
- The reply of a repository `findById` is `Optional<Member>`, not `Member`; write the reply as `Optional[member]` or the row fails the "reply type ok" column.

- A message to `copy : Copy` from the service requires at least a dependency (`..>`) in the class diagram; a solid association is not needed because the service does not keep the copy as a field.

ACCEPTANCE CHECKLIST

- [ ] Three failure scenarios modelled with `break` or `alt`, each ending at the Member with a status and `application/problem+json`; `ApiExceptionHandler` is a lifeline.
- [ ] `ApiExceptionHandler` implemented; the MockMvc test for 409 passes; the controller contains no `try/catch`.
- [ ] `docs/consistency-10.md` has one row per message with the three yes/no columns and a fix for every "no".
- [ ] All fixes applied on the correct side (class diagram updated and marked, or sequence diagrams corrected); all PNGs re-rendered; tag `task-5`.

★

Challenge: reservations with timed expiry, and diagrams generated from tests

Hard **90–120 min · optional, extra credit**

SCENARIO

The library wants online reservations: a member reserves a copy that is currently on loan; when it comes back the member is notified and has 48 hours to collect it, otherwise the reservation expires and the next member in the queue is notified. Fines can be paid online through the provider webhook from Task 3, and the provider is unreliable, so charges are retried. Finally, the team wants the sequence diagram of every service method to be generated from the unit tests so that diagrams never go stale.

REQUIREMENTS

1. `challenge-reservation.mmd`: model *reserve → notify → collect or expire* in one diagram. Participants: `actor Member`, `ReservationController`, `ReservationService`, `ReservationRepository`, `NotificationService`, `ReservationExpiryJob`, `LoanService`. Use `create participant` for the `Reservation`, an async `notifyAvailable(reservation)`, an `alt` with guards `collected within 48h` / `else`, and a `destroy` or `expire()` message when the reservation lapses.
2. Reuse the authentication interaction from Lab-05 with a `ref` fragment (Mermaid: a `Note over` the participants reading `ref: Authenticate (Lab-05)`, PlantUML: `ref over`) instead

of drawing the JWT filter again.

3. Model the online fine payment with retries: a `loop attempt 1..3 while timeout` with a self-call `attempt(request, n)` inside `PaymentGateway`, a `critical` fragment around `fine.markPaid(ref)` in the webhook path, and a `neg` Note stating that a second webhook with the same `ref` must not create a second payment.
4. Sketch the implementation in Java: `@Async NotificationService.notifyAvailable`, a `@Scheduled(fixedDelay = 60_000) ReservationExpiryJob.expireOverdue()` that calls `ReservationService.expire(id)` for every reservation whose `expiresAt` has passed, and `@EnableScheduling`. It must compile; a test with a fixed `Clock` shows that a reservation older than 48 h is expired and the next member is notified.
5. Generate a sequence diagram from a test: add the `MermaidTraceAspect` below (Spring AOP, `spring-boot-starter-aop`) or a plain logging decorator, run `LoanServiceTest.borrowsInDiagramOrder` in a `@SpringBootTest` variant (or wrap the mocks in tracing proxies), and print valid Mermaid lines to `target/generated-diagrams/borrow.mmd`. Render it and compare with `t4-borrow-ecb.mmd` in a short paragraph of `docs/api-10.md`: what the generated diagram shows that the hand-drawn one hides, and vice versa.

SKELETON

```
package edu.itc.library.trace;

/** Prints one Mermaid line per method entry/exit inside edu.itc.library. */
@Aspect
@Component
public class MermaidTraceAspect {
    private static final ThreadLocal<Deque<String>> STACK =
        ThreadLocal.withInitial(ArrayDeque::new);
    private final PrintWriter out;

    public MermaidTraceAspect(@Value("${trace.file}") Path file)
            throws IOException {
        Files.createDirectories(file.getParent());
        out = new PrintWriter(Files.newBufferedWriter(file));
        out.println("sequenceDiagram");
    }

    @Around("within(edu.itc.library..*) && !within(edu.itc.library.trace..*)")
    public Object trace(ProceedingJoinPoint pjp) throws Throwable {
        String callee = pjp.getTarget().getClass().getSimpleName()
                           .replaceAll("\\$\\$.*", "");   // strip proxy suffix
        String caller = STACK.get().isEmpty() ? "Test" : STACK.get().peek();
        String label = pjp.getSignature().getName() + "(" + args(pjp) + ")";
        out.printf("  %s->>+%s: %s%n", caller, callee, label);
```

```
            STACK.get().push(callee);
            try {
                Object result = pjp.proceed();
                out.printf("  %s-->>-%s: %s%n", callee, caller, describe(result));
                return result;
            } catch (Throwable t) {
                out.printf("  %s-->>-%s: throw %s%n", callee, caller,
                           t.getClass().getSimpleName());
                throw t;
            } finally {
                STACK.get().pop();
                out.flush();
            }
        }

        private static String args(ProceedingJoinPoint pjp) {
            return Arrays.stream(pjp.getArgs()).map(MermaidTraceAspect::describe)
                         .collect(Collectors.joining(", "));
        }
        private static String describe(Object o) {
            if (o == null) return "void";
            if (o instanceof Optional<?> opt)
                return opt.map(v -> "Optional[" + describe(v) +
    "]").orElse("empty");
            if (o instanceof Collection<?> c) return "List(" + c.size() + ")";
            return o.getClass().getSimpleName();
        }
    }


    @Component
    public class ReservationExpiryJob {
        private final ReservationRepository reservations;
        private final ReservationService service;
        private final Clock clock;
        // ...
        @Scheduled(fixedDelay = 60_000)
        public void expireOverdue() {
            for (Reservation r : reservations.findNotifiedBefore(Instant.now(clock)

    .minus(Duration.ofHours(48)))) {
                service.expire(r.getId());        // expire, then notify next in
    queue
            }
        }
    }
```

EXPECTED OUTPUT

```
target/generated-diagrams/borrow.mmd
sequenceDiagram
  Test->>+LoanService: borrow(Long, List(1))
  LoanService->>+MemberRepository: findById(Long)
  MemberRepository-->>-LoanService: Optional[Member]
  LoanService->>+CopyRepository: findByBarcode(String)
  CopyRepository-->>-LoanService: Optional[Copy]
  LoanService->>+LoanRepository: save(Loan)
  LoanRepository-->>-LoanService: Loan
  LoanService->>+ReservationRepository: findActiveByCopy(Copy)
  ReservationRepository-->>-LoanService: empty
  LoanService-->>-Test: List(1)

$ ./mvnw -q test
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
```

Show hints

- Spring AOP only intercepts calls between beans, so the trace shows bean-to-bean messages but not `copy.markBorrowed()` on an entity. That is exactly the difference to write up; entity calls can be added with AspectJ compile-time weaving if you are curious.
- Mocks are not Spring beans. For the generated diagram either use `@SpringBootTest` with `@MockitoBean` collaborators (the aspect then sees the service only) or wrap each mock with `Mockito.mock(Type.class, withSettings().defaultAnswer(tracingAnswer))` and print from the `Answer`.
- For timed expiry, a UML duration constraint `{0..48h}` between `notifyAvailable` and `collect` is the formal notation; in Mermaid write it as a Note.
- The `critical` fragment around `markPaid` maps to `@Transactional` plus a unique constraint on `paymentRef`; the `neg` fragment maps to a test that sends the same webhook twice and asserts one payment.
- `@Scheduled` methods run on the scheduler thread; draw `ReservationExpiryJob` as a lifeline whose first message has no sender (a *found* message from a timer). In Mermaid, start it from a Note.

ACCEPTANCE CHECKLIST

- [ ] Reservation diagram covers reserve, async notify, collect-or-expire with the 48 h constraint, a `ref` to Authenticate and a scheduler-triggered expiry.
- [ ] Payment flow shows the retry `loop` with a self-call, a `critical` around `markPaid` and a `neg` statement; the idempotency test passes.
- [ ] `@Async` + `@Scheduled` sketch compiles; the fixed-clock expiry test passes and verifies the next member is notified.

- [ ] `borrow.mmd` is generated by running a test, renders without errors, and the comparison paragraph is in `docs/api-10.md`.

Grading rubric

Component	Points	Full marks when…
Task 1 · Borrow happy path	10	Correct lifelines and order, balanced activations, replies to the caller with values, message names from the class diagram.
Task 2 · Fragments	15	Two `alt`, one `loop`, one `opt` with boolean guards matching Lab-09; abort-or-skip decision documented.
Task 3 · Async, creation, callbacks	20	Create message for `Fine`, async notice with no reply, `par`, webhook as a new message; Java compiles with `@Async` and the webhook endpoint.
Task 4 · ECB and API design	20	Stereotyped diagram obeying ECB rules, API table and signatures derived from messages, `LoanService.borrow` mirrors the diagram, three Mockito tests with `InOrder`.
Task 5 · Error flows and consistency	25	Three failure scenarios to `ProblemDetail`, handler and MockMvc test, complete consistency table with every mismatch fixed on the right side.
Diagram / code quality	10	Source text committed next to every PNG, seven or fewer lifelines per diagram, Notes explain decisions, tags per task, no UI details in design diagrams.
★ Challenge (extra credit)	+20	All four acceptance items ticked; generated diagram renders; comparison paragraph is insightful.
Total	**100 (+20)**	

Automatic deductions: −5 per reply arrow that returns to a lifeline other than the caller; −5 per message whose name is not an operation of the receiver's class (after Task 5); −10 if a boundary sends a message to a repository or entity; −5 per fragment guard that is not a boolean expression; −10 for a diagram submitted as PNG only, without its source text; −10 if the controller contains `try/catch` or business rules.

Submission

1. Repository layout:

```
library-model/
├── README.md                              ← how to render the diagrams, tool
used
├── diagrams/
│   ├── 08-classes/                        ← updated in Task 5, changes marked
│   └── 10-sequence/
│       ├── t1-borrow-happy.mmd  t1-borrow-happy.png
│       ├── t2-borrow-fragments.mmd  t2-borrow-fragments.png
│       ├── t3-return-with-fine.mmd  t3-return-with-fine.png
│       ├── t4-borrow-ecb.mmd  t4-borrow-ecb.png
│       ├── t5-borrow-errors.mmd  t5-borrow-errors.png
│       └── challenge-reservation.mmd  challenge-reservation.png   ←
challenge only
├── docs/
│   ├── api-10.md                          ← endpoints, signatures, status
table, comparison paragraph
│   └── consistency-10.md                  ← consistency table
└── library/                               ← Spring Boot project
    ├── pom.xml · mvnw · .mvn/
    └── src/
        ├── main/java/edu/itc/library/  loan/ · notification/ · payment/ ·
web/ · trace/
        └── test/java/edu/itc/library/  LoanServiceTest ·
BorrowControllerTest · …
```

2. `cd library && ./mvnw -q verify` must succeed from a clean checkout on JDK 25; every `.mmd` must render with `npx @mermaid-js/mermaid-cli -i <file> -o /tmp/x.png` without errors (the grader runs both).
3. Tags `task-1` … `task-5` (and `challenge`) must exist. Push branch `lab-10` and open a merge request titled `Lab-10 – <your name>`.
4. Paste the PNG of `t4-borrow-ecb` and the consistency summary line (messages, mismatches, fixes) into the merge request description.

Back to Chapter 10 slides | Retake the chapter quiz | ← Lab-09